

dependent. Its lack of dependents gives the component no reason not to change, and the components that it depends on may give it ample reason to change.

In contrast, when the I metric is equal to 0, it means that the component is depended on by other components ($Fan-in > 0$), but does not itself depend on any other components ($Fan-out = 0$). Such a component is *responsible* and *independent*. It is as stable as it can get. Its dependents make it hard to change the component, and it has no dependencies that might force it to change.

The SDP says that the I metric of a component should be larger than the I metrics of the components that it depends on. That is, I metrics should *decrease* in the direction of dependency.

NOT ALL COMPONENTS SHOULD BE STABLE

If all the components in a system were maximally stable, the system would be unchangeable. This is not a desirable situation. Indeed, we want to design our component structure so that some components are unstable and some are stable. The diagram in [Figure 14.8](#) shows an ideal configuration for a system with three components.

The changeable components are on top and depend on the stable component at the bottom. Putting the unstable components at the top of the diagram is a useful convention because any arrow that points *up* is violating the SDP (and, as we shall see later, the ADP).

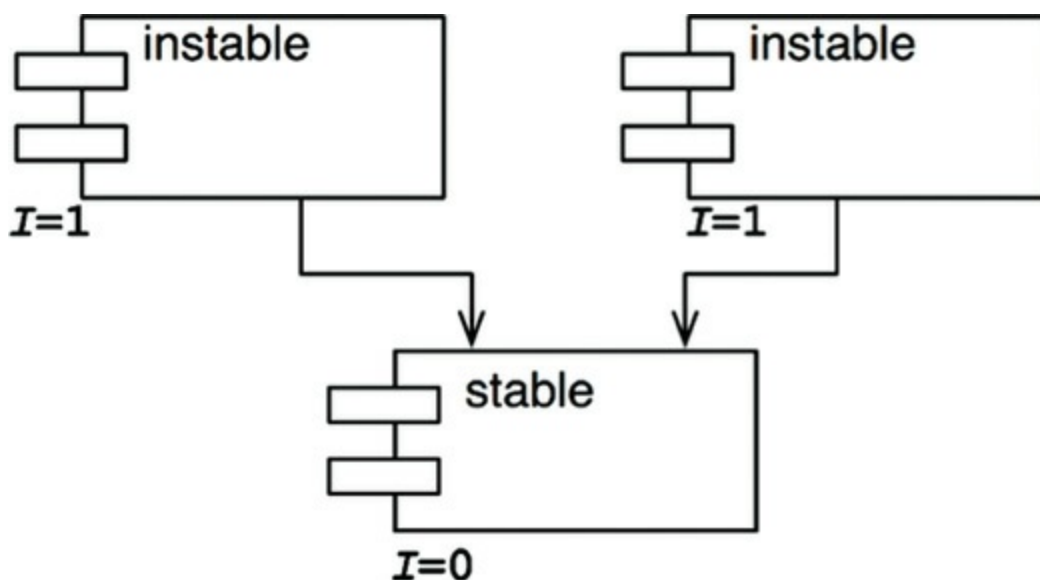


Figure 14.8 An ideal configuration for a system with three components

The diagram in [Figure 14.9](#) shows how the SDP can be violated.

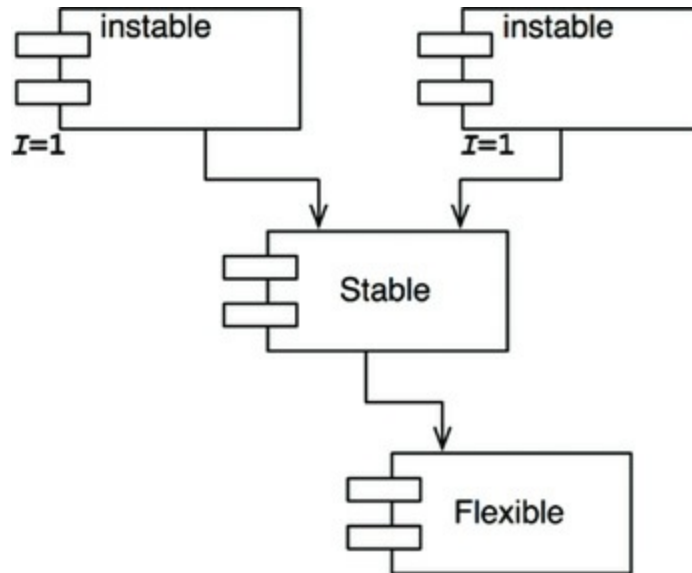


Figure 14.9 SDP violation

`Flexible` is a component that we have designed to be easy to change. We want `Flexible` to be unstable. However, some developer, working in the component named `Stable`, has hung a dependency on `Flexible`. This violates the SDP because the I metric for `Stable` is much smaller than the I metric for `Flexible`. As a result, `Flexible` will no longer be easy to change. A change to `Flexible` will force us to deal with `Stable` and all its dependents.

To fix this problem, we somehow have to break the dependence of `Stable` on `Flexible`. Why does this dependency exist? Let's assume that there is a class `c` within `Flexible` that another class `u` within `Stable` needs to use ([Figure 14.10](#)).

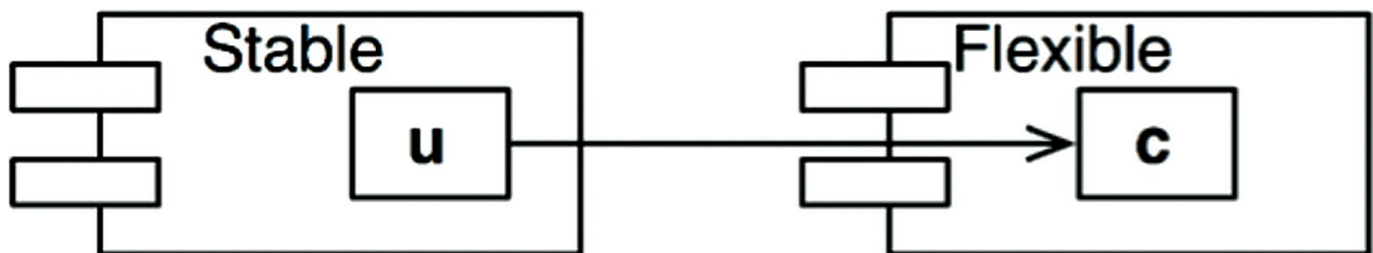


Figure 14.10 `u` within `Stable` uses `c` within `Flexible`

We can fix this by employing the DIP. We create an interface class called `us` and put it in a component named `UServer`. We make sure that this interface declares all the methods that `u` needs to use. We then make `c` implement this interface as shown in [Figure 14.11](#). This breaks the dependency of `Stable` on `Flexible`, and forces both